

Лабораторная работа №8

Тема: Асинхронная обработка событий с использованием RabbitMQ

Проект: D:\fastapi-newyear

Документ описывает добавленный функционал RabbitMQ, SMTP-уведомлений, retry/DLQ, идемпотентности, файлы проекта, ключевые строки и сценарии тестирования.

1. Что добавлено

- ? RabbitMQ 3.12 Management в Docker окружении.
- ? Mailpit как локальный SMTP-сервер для реальной SMTP-отправки и проверки писем через веб-интерфейс.
- ? Модуль очередей app/common/queue с единой точкой подключения к RabbitMQ.
- ? Модуль notifications с producer, background consumer и email-сервисом.
- ? Публикация события user.registered после успешной регистрации пользователя.
- ? Ручные подтверждения ack/nack, retry до 3 попыток и Dead Letter Queue.
- ? Идемпотентность обработки через Redis-ключи wp:events:processed:{eventId}.
- ? Проверка SMTP-конфигурации при старте и перезапуск приложения при критических ошибках consumer.

2. Общая схема работы

Синхронная часть: клиент отправляет POST /auth/register. AuthService валидирует данные, создаёт пользователя и сессию, затем публикует JSON-событие в RabbitMQ exchange app.events с routing key user.registered. Клиент получает HTTP 201 Created.

Асинхронная часть: фоновый consumer слушает очередь wp.auth.user.registered, получает сообщение, проверяет eventId, отправляет приветственное письмо через SMTP, сохраняет Redis-маркер идемпотентности и отправляет ack. Если SMTP недоступен, consumer делает nack(requeue=True) до лимита попыток. После 3 неудач сообщение уходит в wp.auth.user.registered.dlq.

```
POST /auth/register
-> AuthService.register
-> publish_user_registered_event
-> RabbitMQ exchange app.events
-> queue wp.auth.user.registered
-> background consumer
-> SMTP welcome email
-> ack
```

Ошибка SMTP:

```
-> nack(requeue=True) до 3 попыток
-> nack(requeue=False)
-> wp.auth.user.registered.dlq
```

3. Файлы и ответственность

Файл	За что отвечает
docker-compose.yml	Добавляет rabbitmq и mailpit, healthcheck RabbitMQ, порты 5672/15672 и 1025/8025, volume rabbitmq_data, зависимости app от RabbitMQ/Mailpit.
.env	Хранит настройки RabbitMQ и SMTP: логин, пароль, exchange, DLQ, очереди, SMTP host/port/from/login URL.
requirements.txt	Добавляет pika - Python клиент RabbitMQ.

app/common/config.py	Описывает настройки RabbitMQ/SMTP и валидирует SMTP-конфигурацию при старте.
app/common/queue/rabbitmq.py	Единый сервис RabbitMQ: подключение по паролю, exchange/queue/DLQ topology, publish, consume, ack, nack.
app/modules/auth/service.py	После успешной регистрации пользователя вызывает публикацию события user.registered.
app/modules/notifications/events.py	Producer: формирует безопасное JSON-событие без паролей и токенов, публикует его в RabbitMQ.
app/modules/notifications/consumer.py	Consumer: фоновый воркер, retry, DLQ, Redis idempotency, ack/nack, restart при критических ошибках.
app/modules/notifications/email_service.py	SMTP-отправка приветственного письма в plain text и HTML, логин в SMTP при наличии credentials.
app/main.py	Startup/shutdown lifecycle: проверка SMTP, RabbitMQ setup, запуск и остановка consumer, настройка логирования.

4. Основные места в коде по строкам

Файл и строки	Что реализовано
docker-compose.yml:56-74	Сервис RabbitMQ 3.12-management-alpine, пользователь и пароль из .env, порты 5672 и 15672, volume и healthcheck.
docker-compose.yml:76-82	Mailpit как локальный SMTP-сервер и UI для проверки писем.
docker-compose.yml:97-100	app ждёт RabbitMQ healthcheck и старт Mailpit.
.env:50-59	RabbitMQ host/port/user/pass, exchange app.events, DLX app.dlx, queues и лимиты retry/failure.
.env:61-67	SMTP host/port/from и ссылка входа в письмо.
requirements.txt:12	Зависимость pika>=1.3,<2.0.
app/common/config.py:56-65	Настройки RabbitMQ.
app/common/config.py:67-73	Настройки SMTP.
app/common/config.py:135-146	validate_smtp_config: понятная ошибка, если SMTP настроен неполно.
app/main.py:16-17	Логирование приложения, приглушение шумных логов pika.
app/main.py:48-54	Startup: валидация SMTP, RabbitMQ setup, запуск consumer.
app/main.py:59	Shutdown: остановка consumer.
app/common/queue/rabbitmq.py:25-35	Подключение к RabbitMQ с login/password из .env.
app/common/queue/rabbitmq.py:38-64	Durable direct exchange app.events, DLX app.dlx, основная очередь и DLQ, binding по user.registered.
app/common/queue/rabbitmq.py:89-99	Публикация persistent JSON-сообщения с delivery_mode=2 и confirm_delivery.
app/common/queue/rabbitmq.py:115-138	Consumer callback, JSON decode, invalid JSON -> DLQ, auto_ack=False.
app/common/queue/rabbitmq.py:149-155	Методы ack и nack.
app/modules/auth/service.py:31, 135	Интеграция producer в регистрацию: publish_user_registered_event(user).
app/modules/notifications/events.py:12-29	Формирование события eventId/eventType/timestamp/payload/metadata и отправка в exchange.
app/modules/notifications/events.py:34-36	Лог публикации без чувствительных данных.
app/modules/notifications/consumer.py:26-30	Валидация email config и старт фонового thread consumer.
app/modules/notifications/consumer.py:41-57	Глобальная защита consumer: reconnect, лимит падений, os_exit(1) для Docker restart.
app/modules/notifications/consumer.py:65-76	Получение eventId, попытки, проверка типа события и idempotency.
app/modules/notifications/consumer.py:80-89	Отправка письма, Redis mark processed, ack.
app/modules/notifications/consumer.py:91-98	Retry через nack(requeue=True), после лимита nack(requeue=False) -> DLQ.
app/modules/notifications/consumer.py:100-123	Redis ключи wp.events:processed:{eventId} и wp.events:attempts:{eventId} с TTL 24 часа.
app/modules/notifications/email_service.py:21-26	Тема, From/To, plain text и HTML alternative.
app/modules/notifications/email_service.py:28-39	SMTP/SMTP_SSL, login при наличии SMTP_USER/SMTP_PASS, send_message.
app/modules/notifications/email_service.py:42-66	Шаблоны письма: обращение по имени, подтверждение регистрации, ссылка входа.

5. Формат сообщения

```
{
  "eventId": "uuid",
  "eventType": "user.registered",
  "timestamp": "2026-05-01T20:00:00Z",
  "payload": {
    "userId": "uuid",
```

```

    "email": "user@example.com",
    "displayName": "User Name"
  },
  "metadata": {
    "attempt": 1,
    "sourceService": "auth-service"
  }
}

```

В сообщении нет паролей, access/refresh токенов, JWT, refresh token hash и других секретов. Передаются только данные, необходимые для отправки welcome email.

6. Сценарии тестирования

6.1. Запуск

```

docker compose up --build -d
docker compose ps
docker compose logs --tail=80 app

```

RabbitMQ UI: <http://localhost:15672>. Логин student, пароль из RABBITMQ_PASS. Mailpit UI: <http://localhost:8025>.

6.2. Публикация события при регистрации

```

$email = "lab8.$([guid]::NewGuid().ToString('N'))@example.com"
@{ email = $email; password = "Winter2026" } | ConvertTo-Json -Compress | Set-Content -Encoding UTF8 -NoNewline "$env:TEMP\lab8-register.json"
curl.exe -i -H "Content-Type: application/json" --data-binary "@$env:TEMP\lab8-register.json"
http://localhost:4200/auth/register
docker compose logs --tail=80 app

```

Ожидается: HTTP 201 Created, в логгах Published RabbitMQ event и User registered event published.

6.3. Потребление и отправка email

```

docker exec wp_labs_rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
curl.exe -s http://localhost:8025/api/v1/messages

```

Ожидается: очередь wp.auth.user.registered имеет 0 ready и 0 unacknowledged; в Mailpit есть письмо Welcome to Holiday Prep.

6.4. Проверка retry и DLQ

```

docker compose stop mailpit
# выполнить POST /auth/register с новым email
Start-Sleep -Seconds 55
docker exec wp_labs_rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
docker compose start mailpit

```

Ожидается: после 3 неудачных SMTP-попыток сообщение окажется в wp.auth.user.registered.dlq.

6.5. Проверка идиempотентности

Дважды опубликуйте одинаковый JSON с одним eventId в exchange app.events и routing key user.registered. Первое сообщение отправит письмо и поставит Redis key wp:events:processed:{eventId}. Второе сообщение будет ask без повторной отправки письма.

```

docker exec wp_labs_redis redis-cli -a redis_secure_password --scan --pattern "wp:events:processed:*"

```

7. Ответы на контрольные вопросы

1. В чем принципиальная разница между синхронным HTTP-запросом и асинхронной отправкой сообщения в очередь?

При синхронном HTTP клиент ждёт ответ прямо сейчас: вызывающая сторона и получатель связаны по времени и доступности. При отправке в очередь producer кладёт сообщение в брокер и продолжает работу, а

consumer обрабатывает его позже. Это снижает связанность, помогает переживать временную недоступность SMTP/worker и разгружает основной HTTP-запрос.

2. Что такое exchange в RabbitMQ? Чем direct отличается от fanout и topic?

Exchange — маршрутизатор сообщений. Producer публикует сообщение не напрямую в очередь, а в exchange, а exchange решает, в какие очереди отправить сообщение. Direct отправляет по точному совпадению routing key и binding key. Fanout игнорирует routing key и рассылает во все привязанные очереди. Topic маршрутизирует по шаблонам с точечной нотацией, например user.* или user.#.

3. Зачем нужны acknowledgements и что произойдет, если потребитель упадет до ack?

Ack подтверждает брокеру, что сообщение успешно обработано и его можно удалить из очереди. Если consumer получил сообщение, но упал до ack, RabbitMQ вернёт сообщение в очередь и доставит его снова другому или восстановленному consumer. Это защищает от потери сообщений при сбоях.

4. Что такое Dead Letter Queue и в каких сценариях ее использование оправдано?

DLQ — отдельная очередь для сообщений, которые не удалось обработать штатно: исчерпаны retries, сообщение повреждено, истёк TTL, получен reject/nack без requeue. Она оправдана, когда сообщение нельзя бесконечно гонять по основной очереди, но его нужно сохранить для анализа, ручного исправления или повторной обработки.

5. Почему важно обеспечивать идемпотентность обработчиков сообщений? Как это реализовать на практике?

В очередях возможны повторные доставки: consumer мог выполнить действие, но упасть до ack, или producer мог отправить событие повторно. Идемпотентность гарантирует, что повторная обработка не создаст дубликаты писем, платежей или записей. Практически это делается через уникальный eventId и хранилище processed events: Redis SET NX с TTL, уникальный индекс в БД или outbox/inbox таблицы.

6. Какие данные нельзя передавать в сообщениях очереди и почему?

Нельзя передавать пароли, JWT/access/refresh токены, refresh token hash, OAuth secrets, API keys, персональные данные сверх необходимого минимума. Сообщения могут храниться на диске брокера, попадать в DLQ, логи, мониторинг и резервные копии. Поэтому payload должен содержать только минимальные данные для обработки события.

7. Как обеспечить надежность доставки сообщений при перезапуске брокера или потребителя?

Нужно объявлять exchange и queues как durable, публиковать persistent сообщения delivery_mode=2, использовать publisher confirms, ack только после успешной обработки, настраивать retry и DLQ, запускать consumer при старте приложения, хранить состояние идемпотентности и использовать restart policy контейнеров. В проекте это реализовано через durable topology, persistent publish, manual ack/nack, Redis idempotency и Docker restart.

8. В чем преимущества и недостатки использования брокера сообщений по сравнению с прямыми вызовами между сервисами?

Преимущества: слабая связанность, устойчивость к временным сбоям, сглаживание пиков нагрузки, retry/DLQ, возможность нескольких consumer. Недостатки: больше инфраструктуры, сложнее отладка, eventual consistency вместо мгновенного результата, необходимость идемпотентности, мониторинга очередей и контроля DLQ.

8. Итоговая проверка, выполненная в проекте

? docker compose up --build -d app — успешно.

? POST /auth/register — 201 Created.

? В логах есть publish, receive, sending welcome email, success.

? RabbitMQ queue wp.auth.user.registered после обработки: 0 ready, 0 unacknowledged.

? Mailpit получил письмо Welcome to Holiday Prep с plain text и HTML содержимым.

? При выключенном SMTP после 3 попыток сообщение попало в wp.auth.user.registered.dlq.

? Дублирующее событие с одинаковым eventId не отправило второе письмо благодаря Redis idempotency.